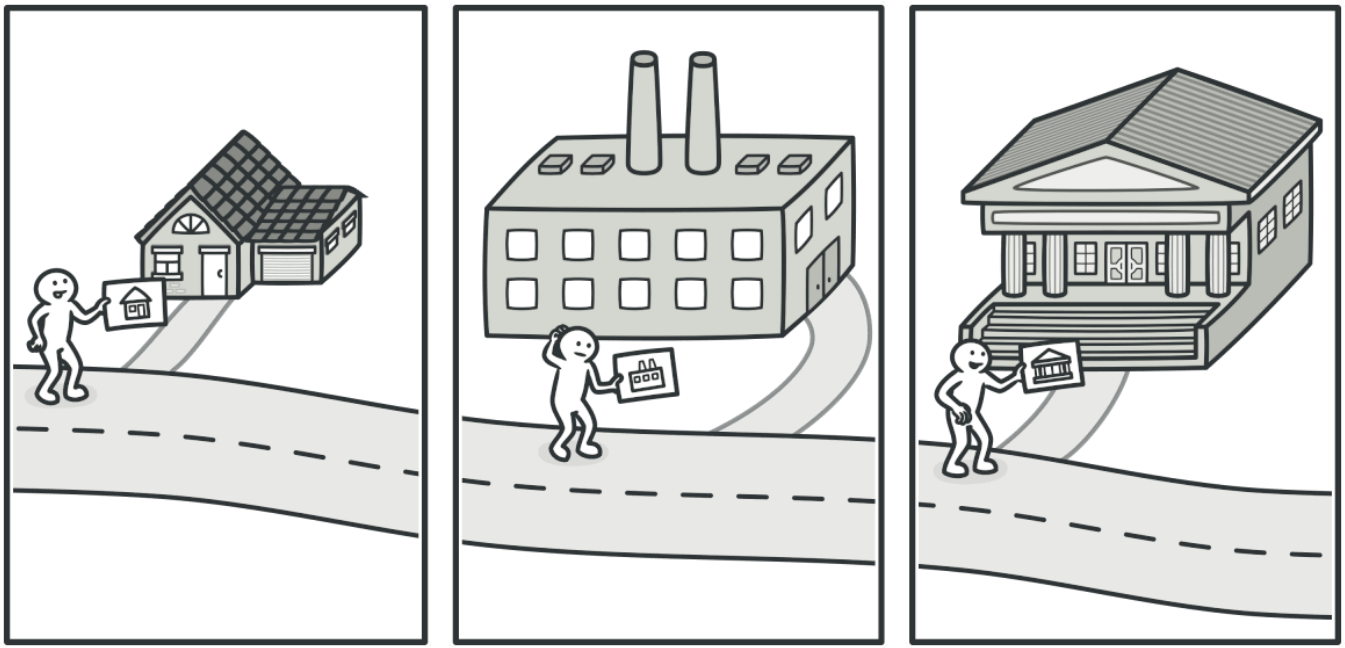
Поведенческие паттерны. Посетитель

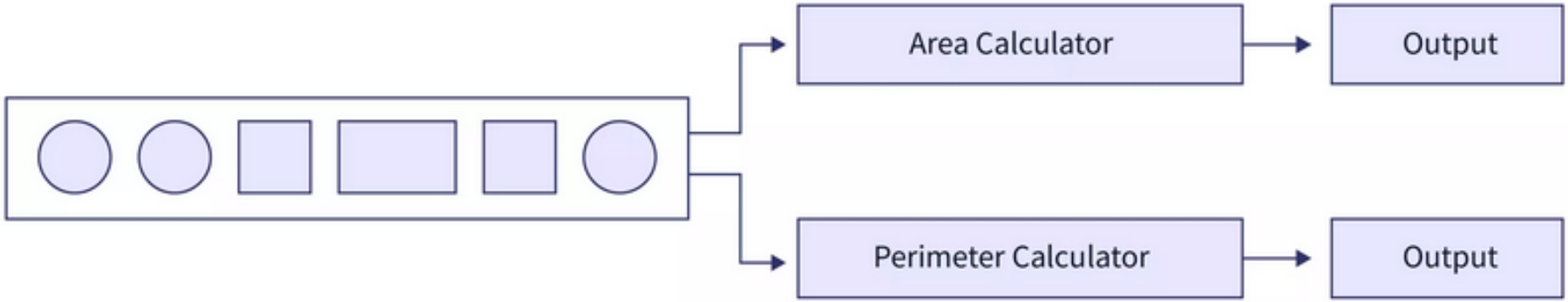


Паттерн Посетитель
Visitor



Посетитель — это поведенческий паттерн, который позволяет добавить новую операцию для иерархии классов, не изменяя код этих классов.

Проблема



Имеется список, состоящий из множества **фигур** (**квадрат, круг, прямоугольник** и т.д.). Необходимо вычислить площадь и периметр этих фигур.

Простой способ - использовать методы **area()** и **perimeter()** в соответствующих классах фигур. При этом способе возникает тесная связь *алгоритма* (вычисление площади и периметра) с *объектами* (**квадрат, круг, прямоугольник** и т.д.), с которыми они работают.

С помощью шаблона проектирования **Посетитель** можно устранить эту тесную связь, разделив *алгоритмы* и *объекты*, с которыми они работают.

В будущем, если к объектам будут добавлены какие-либо новые алгоритмы, их можно будет добавлять отдельно, не затрагивая базовые объекты. Так реализуется принцип открытости/закрытости (т.е. классы должны быть открыты для расширения, но закрыты для модификации).

Решение

Паттерн **Посетитель** предлагает разместить новое поведение в отдельном классе, вместо того чтобы множить его сразу в нескольких классах.

Объекты, с которыми должно было быть связано поведение, не будут выполнять его самостоятельно. Клиент передает эти объекты в методы **Посетителя**.

Код поведения может отличаться для объектов разных классов, поэтому и методов у посетителя должно быть несколько.

Названия и принцип действия этих методов будут схожи, но основное отличие будет в типе принимаемого в параметрах объекта.

Пример

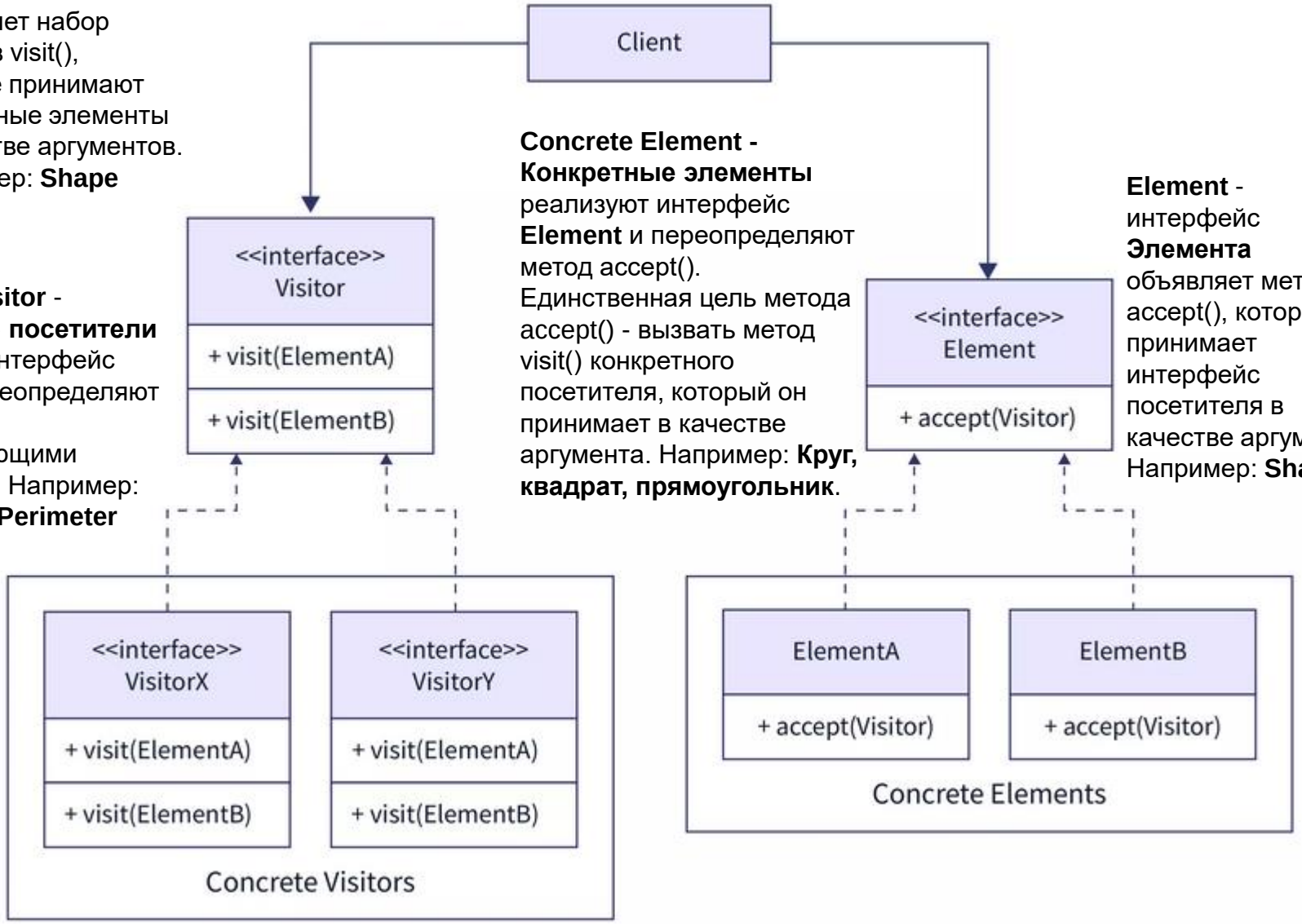
Client - Клиент отвечает за создание конкретных экземпляров visitor и передачу их методу accept() для конкретных экземпляров **Element**.

Visitor - интерфейс **Посетителя** объявляет набор методов visit(), которые принимают конкретные элементы в качестве аргументов. Например: **Shape Visitor**

Concrete Visitor - **Конкретные посетители** реализуют интерфейс **Visitor** и переопределяют метод visit() соответствующими операциями. Например: **AreaVisitor, Perimeter Visitor**

Concrete Element - Конкретные элементы реализуют интерфейс **Element** и переопределяют метод accept(). Единственная цель метода accept() - вызвать метод visit() конкретного посетителя, который он принимает в качестве аргумента. Например: **Круг, квадрат, прямоугольник**.

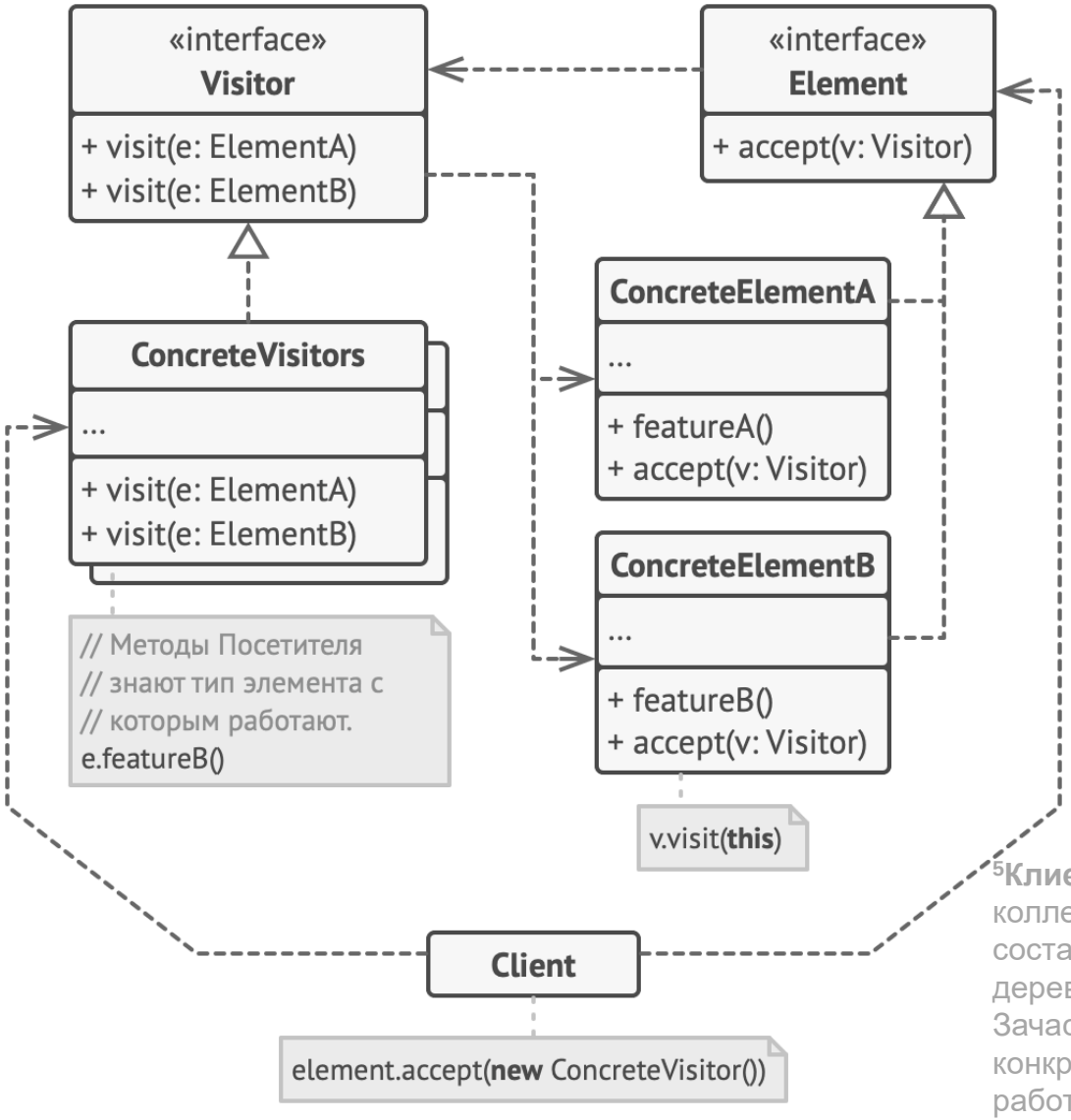
Element - интерфейс **Элемента** объявляет метод accept(), который принимает интерфейс посетителя в качестве аргумента. Например: **Shape**



Структура паттерна

[1]**Посетитель** описывает общий интерфейс для всех типов посетителей. Он объявляет набор методов, отличающихся типом входящего параметра, которые нужны для запуска операции для всех типов конкретных элементов. В языках, поддерживающих перегрузку методов, эти методы могут иметь одинаковые имена, но типы их параметров должны отличаться.

[2]**Конкретные посетители** реализуют какое-то особенное поведение для всех типов элементов, которые можно подать через методы интерфейса посетителя.

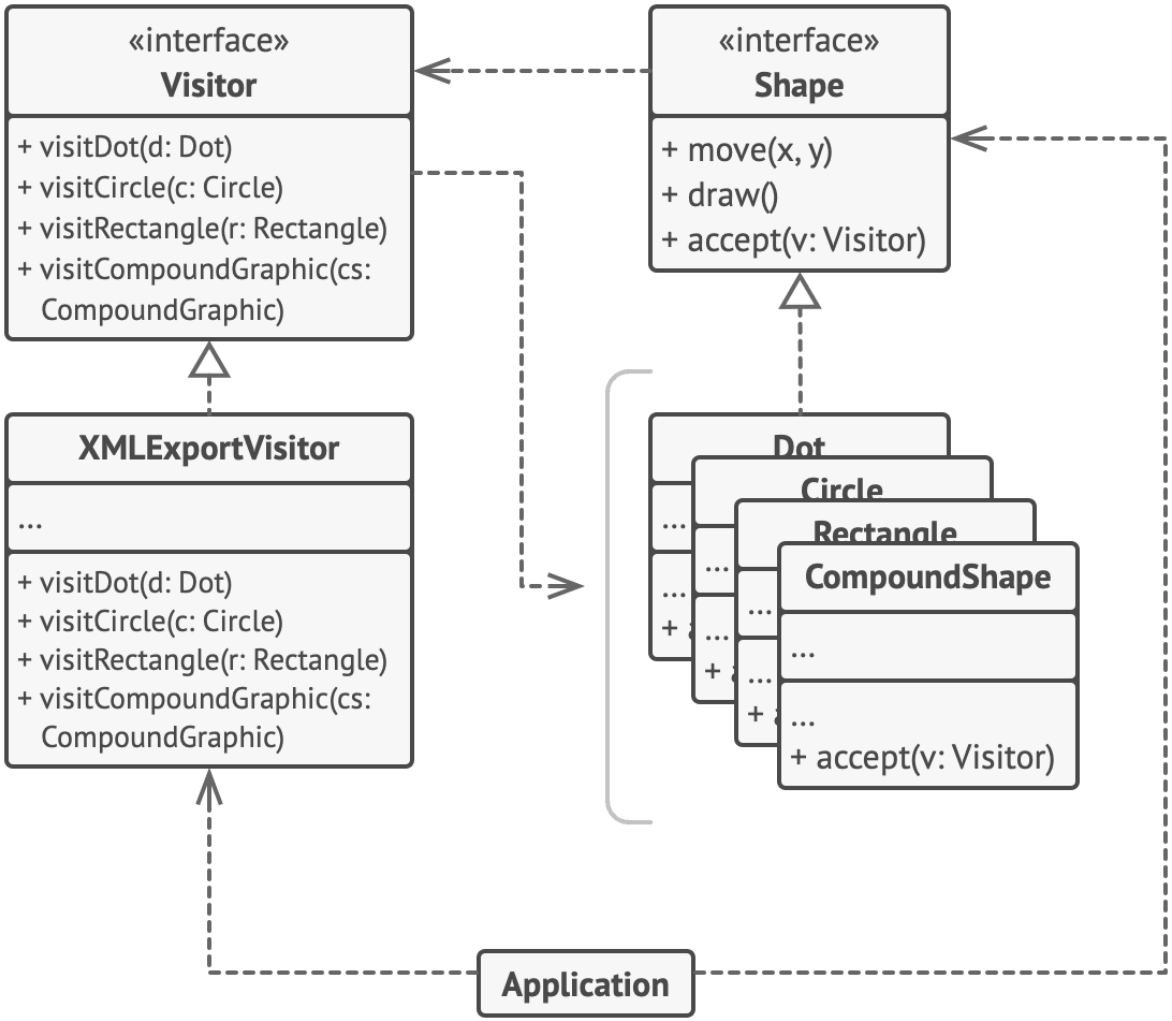


[3]**Элемент** описывает метод принятия посетителя. Этот метод должен иметь единственный параметр, объявленный с типом интерфейса посетителя..

[4]**Конкретные элементы** реализуют методы принятия посетителя. Цель этого метода — вызвать тот метод посещения, который соответствует типу этого элемента. Так посетитель узнает, с каким именно элементом он работает.

[5]**Клиентом** зачастую является коллекция или сложный составной объект, например, дерево Компоновщика. Зачастую клиент не привязан к конкретным классам элементов, работая с ними через общий интерфейс элементов.

Пример организации экспорта объектов в XML через отдельный класс-посетитель.



Пример

```
// Сложная иерархия элементов.
interface Shape is
    method move(x, y)
    method draw()
    method accept(v: Visitor)

// Метод принятия посетителя должен бы
// элементе, а не только в базовом кла
// определить, какой метод посетителя
// знаете тип элемента.
class Dot implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitDot(this)

class Circle implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitCircle(this)

class Rectangle implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitRectangle(this)

class CompoundShape implements Shape is
    // ...
    method accept(v: Visitor) is
        v.visitCompoundShape(this)
```

```
// Интерфейс посетителей должен содержать методы посещения
// каждого элемента. Важно, чтобы иерархия элементов менялась
// редко, так как при добавлении нового элемента придётся менять
// всех существующих посетителей.
interface Visitor is
    method visitDot(d: Dot)
    method visitCircle(c: Circle)
    method visitRectangle(r: Rectangle)

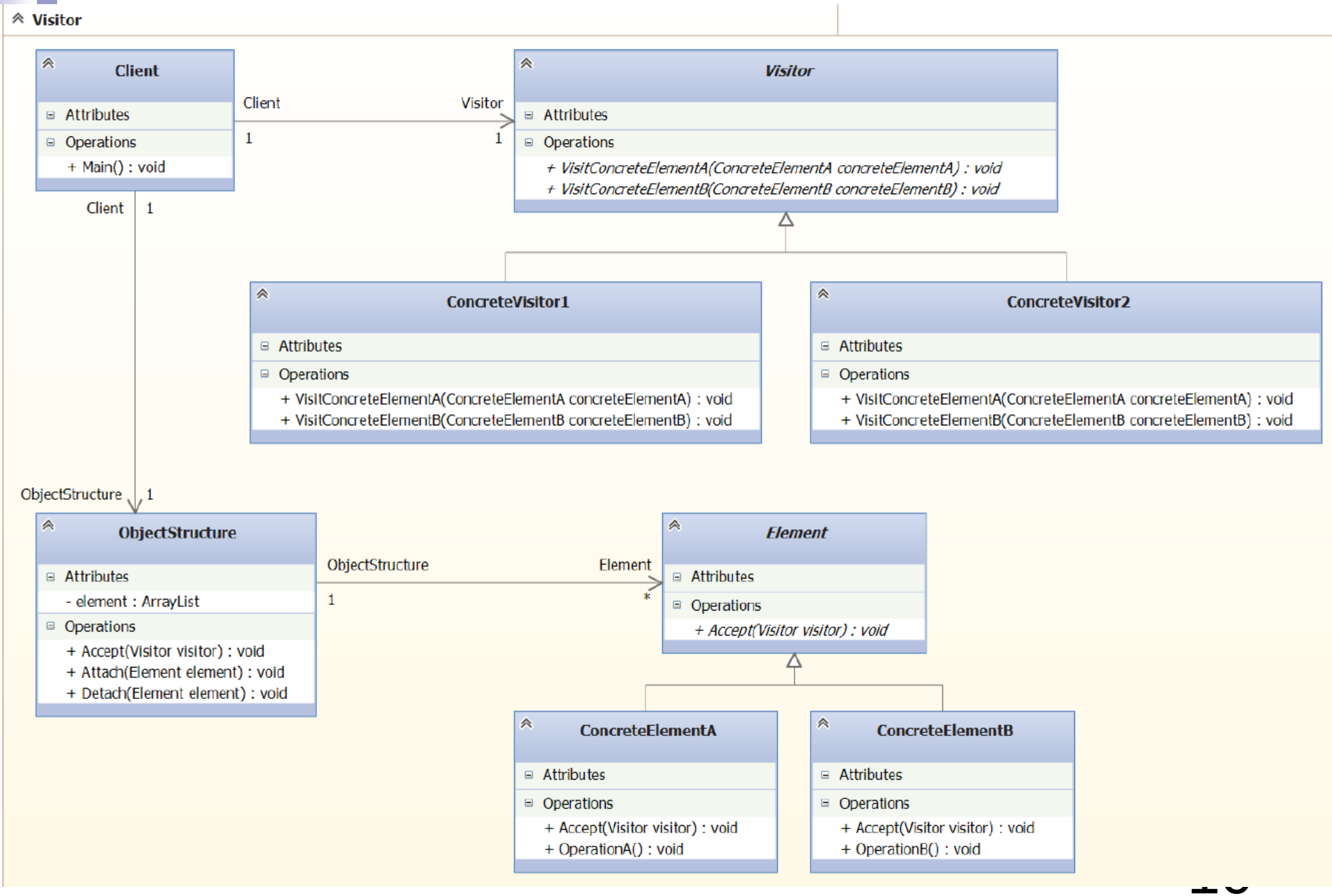
// Конкретный посетитель реализует одну операцию для всей
// иерархии элементов. Новая операция = новый посетитель.
// Посетитель выгодно применять, когда новые элементы
// добавляются очень редко, а новые операции — часто.
class XMLExportVisitor implements Visitor is
    method visitDot(d: Dot) is
        // Экспорт id и координат центра точки.

    method visitCircle(c: Circle) is
        // Экспорт id, кординат центра и радиуса окружности.

    method visitRectangle(r: Rectangle) is
        // Экспорт id, кординат левого-верхнего угла, ширины и
        // высоты прямоугольника.

    method visitCompoundShape(cs: CompoundShape) is
        // Экспорт id составной фигуры, а также списка id
        // подфигур, из которых она состоит.
```

Структура паттерна UML



Структура паттерна на языке C#

```
class Program
{
    static void Main()
    {
        var structure = new ObjectStructure();

        structure.Add(new ConcreteElementA());
        structure.Add(new ConcreteElementB());

        structure.Accept(new ConcreteVisitor1());
        structure.Accept(new ConcreteVisitor2());
    }
}
```

`Visitor` - **Посетитель**:
Предоставляет абстрактный интерфейс (набор методов `VisitConcretElementX`) для работы с объектами класса `ConcreteElementX`. Имя метода `VisitConcretElementX` включает в себя имя класса, экземпляр которого вызывает данный метод.

```
abstract class Visitor
{
    public abstract void VisitElementA(ElementA elementA);
    public abstract void VisitElementB(ElementB elementB);
}
```

`ConcreteVisitor` - **Конкретный посетитель**:
Реализует абстрактный интерфейс, предоставляемый абстрактным классом `Visitor`. Каждая операция `VisitConcretElementX` реализует фрагмент алгоритма, специфичного для каждого отдельного класса `ConcreteElement`.

```
class ConcreteVisitor1 : Visitor
{
    public override void VisitElementA(ElementA elementA)
    {
        elementA.OperationA();
    }

    public override void VisitElementB(ElementB elementB)
    {
        elementB.OperationB();
    }
}
```

```
class ConcreteVisitor2 : Visitor
{
    public override void VisitElementA(ElementA elementA)
    {
        elementA.OperationA();
    }

    public override void VisitElementB(ElementB elementB)
    {
        elementB.OperationB();
    }
}
```

`ObjectStructure` - **Структура объектов**:
Представляет собой набор объектов типа `Element`. Может быть, как обычной коллекцией, так и древообразной структурой.

```
class ObjectStructure
{
    ArrayList elements = new ArrayList();

    public void Add(Element element)
    {
        elements.Add(element);
    }

    public void Remove(Element element)
    {
        elements.Remove(element);
    }

    public void Accept(Visitor visitor)
    {
        foreach (Element element in elements)
            element.Accept(visitor);
    }
}
```

`Element` - элемент:
Предоставляет абстрактный метод `Accept`, который принимает аргумент типа `Visitor`.

```
abstract class Element
{
    public abstract void Accept(Visitor v);
}
```

`ConcreteElement` - **Конкретный элемент**:
Реализует абстрактный метод `Accept`, который принимает аргумент типа `Visitor`.

```
class ElementA : Element
{
    public override void Accept(Visitor v)
    {
        v.VisitElementA(this);
    }

    public void OperationA()
    {
        Console.WriteLine("OperationA");
    }
}
```

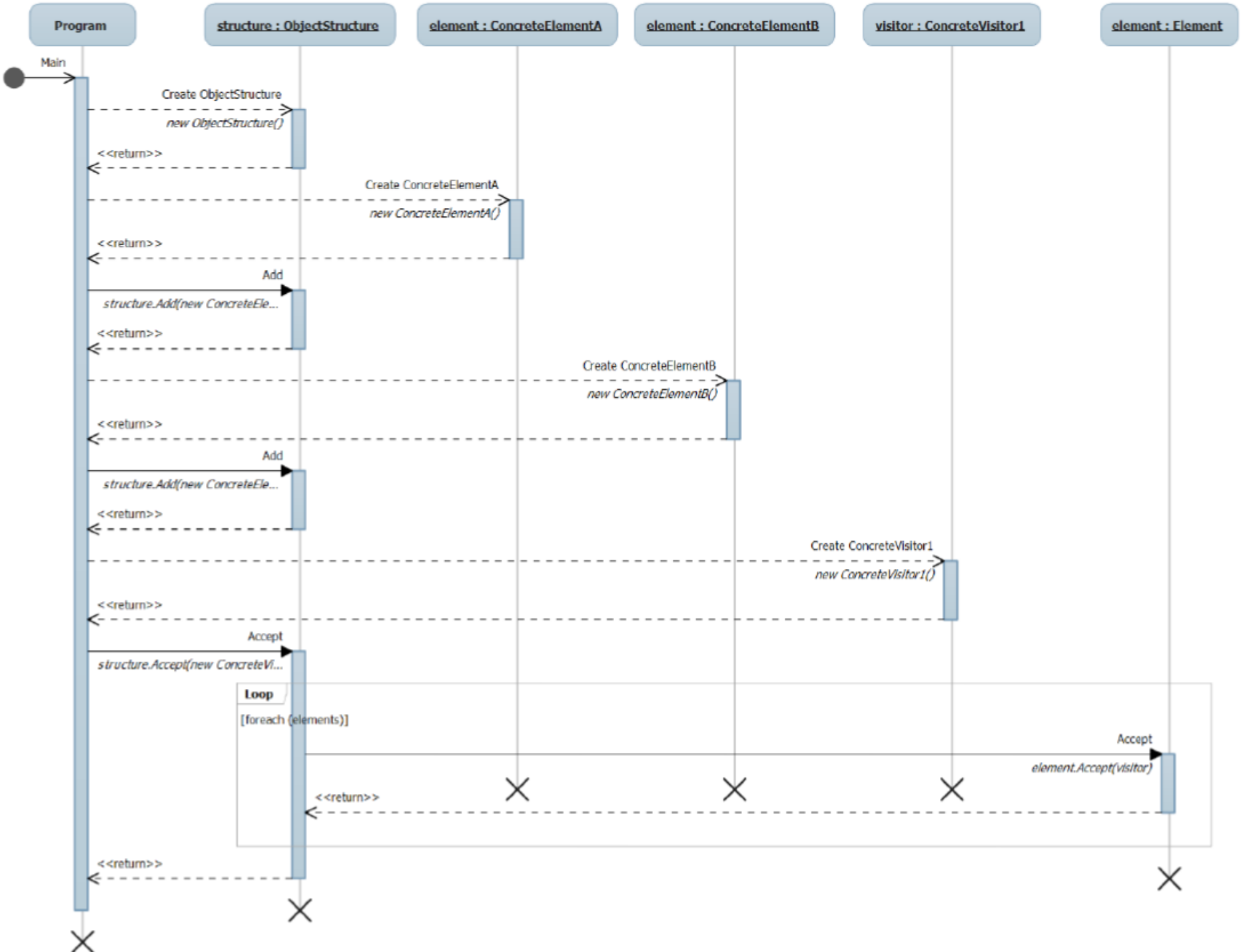
```
class ElementB : Element
{
    public override void Accept(Visitor v)
    {
        v.VisitElementB(this);
    }

    public void OperationB()
    {
        Console.WriteLine("OperationB");
    }
}
```

Отношения между участниками

Отношения между классами

- Конкретные классы ConcreteVisitor связаны связью отношения наследования с абстрактным классом Visitor.
- Конкретные классы ConcreteElement связаны связью отношения наследования с абстрактным классом Element.
- Конкретный класс ObjectStructure связан связью отношения ассоциации с абстрактным классом Element.
- Отношения между объектами
- Клиент использующий паттерн Visitor должен создать экземпляр класса ConcreteVisitor и с его помощью обойти каждый элемент объектной структуры (коллекции).
- При посещении экземпляром класса ConcreteVisitor определенного элемента (экземпляра класса ConcreteElement) из объектной структуры, этот элемент вызывает на посетителе метод VisitConcretElementX, соответствующий классу данного элемента. Элемент передает этому методу себя в качестве аргумента, чтобы посетитель мог получить доступ к членам (состоянию и поведению) данного элемента.


Program
structure : ObjectStructure
element : ConcreteElementA
element : ConcreteElementB
visitor : ConcreteVisitor1
element : Element
Main
Create ObjectStructure
new ObjectStructure()
<<return>>
Create ConcreteElementA
new ConcreteElementA()
<<return>>
Add
structure.Add(new ConcreteEle...
<<return>>
Create ConcreteElementB
new ConcreteElementB()
<<return>>
Add
structure.Add(new ConcreteEle...
<<return>>
Create ConcreteVisitor1
new ConcreteVisitor1()
<<return>>
Accept
structure.Accept(new ConcreteVi...
Loop
[foreach (elements)]
Accept
element.Accept(visitor)
<<return>>
<<return>>

Применение

1. Когда нужно выполнить какую-то операцию над всеми элементами сложной структуры объектов, например, деревом.

Посетитель позволяет применять одну и ту же операцию к объектам различных классов.

2. Когда над объектами сложного контейнера надо выполнять некоторые не связанные между собой операции не вводя их в классы.

Посетитель позволяет извлечь родственные операции из классов, составляющих структуру объектов, поместив их в один класс-посетитель. Если структура объектов является общей для нескольких приложений, то паттерн позволит в каждое приложение включить только нужные операции.

3. Когда новое поведение имеет смысл только для некоторых классов из существующей иерархии.

Посетитель позволяет определить поведение только для этих классов, оставив его пустым для всех остальных.

Шаги реализации

1**. Создать интерфейс посетителя с методами «посещения»** для каждого класса элемента, который существует в программе.

2. **Описать интерфейс элементов**. При работе с уже существующими классами, следует объявить абстрактный метод принятия посетителей в базовом классе иерархии элементов.

3. **Реализовать методы принятия во всех конкретных элементах**. Они должны переадресовывать вызовы тому методу посетителя, в котором тип параметра совпадает с текущим классом элемента.

4. **Иерархия элементов должна знать только о базовом интерфейсе посетителей**. С другой стороны, посетители будут знать обо всех классах элементов.

5. **Для каждого нового поведения создать конкретный класс посетителя**. Приспособить это поведение для работы со всеми типами элементов, реализовав все методы интерфейса посетителей.

В ситуации, когда посетителю нужен будет доступ к приватным полям элементов:

- *раскрыть доступ к этим полям, нарушив инкапсуляцию элементов;*
- *сделать класс посетителя вложенным в класс элемента.*

6. **Клиент будет создавать объекты посетителей**, а затем передавать их элементам, используя метод принятия.

Преимущества и недостатки

+ Упрощает добавление операций, работающих со сложными структурами объектов.
+ Объединяет родственные операции в одном классе.
+ Посетитель может накапливать состояние при обходе структуры элементов.
- Паттерн не оправдан, если иерархия элементов часто меняется.
- Может привести к нарушению инкапсуляции элементов.